UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

# Inheritance and polymorphism

Readings: (  :chapt. 12)

1

## Exemplo: mais figuras geométricas

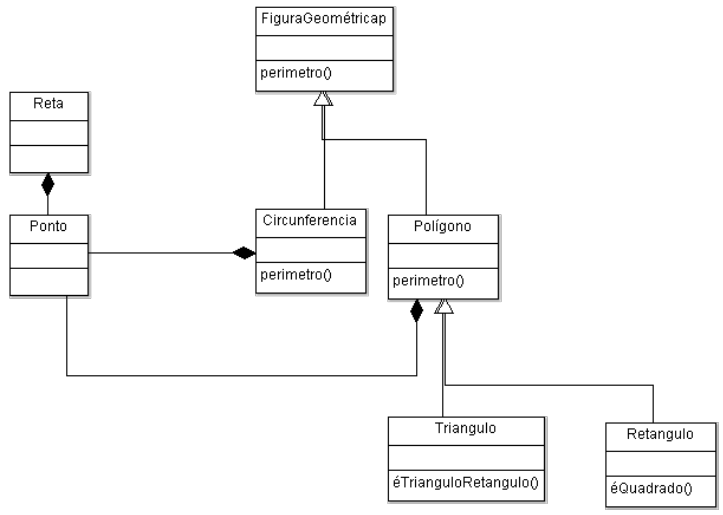
- Para a implementação de um editor gráfico é necessário guardar e manipular figuras geométricas a duas dimensões.

  O editor suporta rectas (definidas por 2 pontos representativos), triângulos (representados por 3 pontos representativos), rectângulos (representados por 4 pontos representativos), polígonos (representados por um número finito de pontos representativos) e circunferências (representadas por centro e um raio).

  Todas as figuras geométricas deverão ser capazes de responder o seu perímetro. Adicionalmente, um rectângulo deverá saber responder se é, ou não, um quadrado e um triângulo deverá saber se é, ou não, um triângulo rectângulo.

2

# An initial (analysis) class diagram



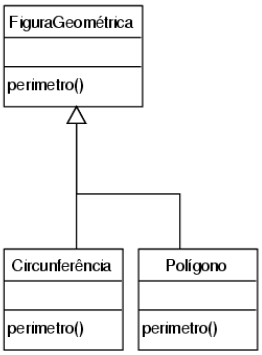

3

# Uma implementação em Java

https://www.programiz.com/online-compiler/9ysu8h9S1dggN

4

# Polymorphism - Example

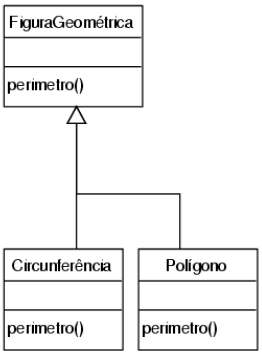



```
public class Poligono extends FiguraGeometrica {
 private int nv;    // number of vertices
 private Ponto[] V;   // vertices
//...
public double perimetro () {
  double sum=V[nv-1].distancia(V[0]);
  for (int i=0; i<nv-1; ++i)
              sum+=V[i].distancia(V[i+1]);
  IO.print("Perimetro de poligono: ");
  return sum;
}

}
```

5

# Polymorphism - Example




```
public class Circunferencia extends FiguraGeometrica {
  private double raio;
  private Ponto centro;

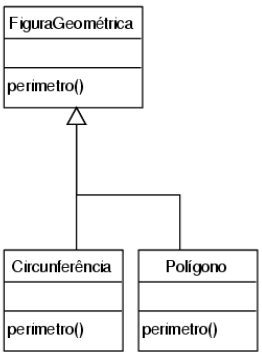
//…

public double perimetro () {
   double d = 2.0*Math.PI*raio;
   IO.print("Perimetro de Circunferencia: ");
   return d;
}

}
```

6

# Polymorphism - Example




```
Queue<FiguraGeometrica> q = new LinkedList<>();

Circunferencia c = new Circunferencia(centro, raio);
Polígono p = new Poligono (arrayDePontos);

q.add(c);
q.add(p);
FiguraGeometrica f;
while(!q.isEmpty()) {
        f = q.remove();
        IO.println(f.perimetro());
}
```

Perimetro de circunferencia: 3.14

Perimetro de poligno: 5

7

# A first notion of Polymorphism

- The primary usage of the (universal subtype) polymorphism is the ability of objects belonging to different types to respond to the same message with type-specific behavior.
- The programmer does not have to know the exact type of the object in advance, the exact behavior being determined at run time.

8

## OOP defining characteristics

- Abstraction of data and code (classes and objects);
- Encapsulation

} ADT

- Inheritance
- (Universal subtype) polymorphism

9

## Encapsulation/information hiding

- In Object-oriented programming, the ability to hide internal detail from client code is referred to as *encapsulation* or *information hiding*

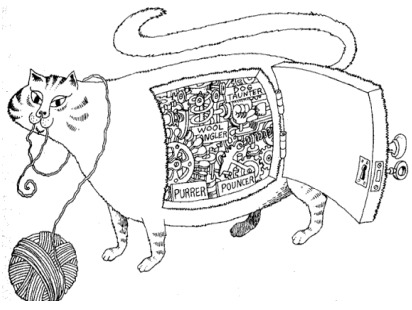



Image from: Grady Booch et al., *Object-oriented Analysis and Design with Application*, Wiley, 2007

10

## Data type – a definition

- A data type is a set of values and a collection of operations on this set of values.

  Example: The *int* data type has the following set of values

  { …, -2, -1, 0, 1, 2, … },

  and operations on these values such as: +, *, etc.



## Abstract Data Type – a definition

- An ADT (abstract data type) is a data type whose values are ***only*** available to clients through the operations in the ADT interface

- In C, a string ***is not*** an ADT. Why?
- and in Java is String an ADT? Why?

## ADT: *Some* advantages

1. Clients of ADT point do not know how a point is represented, therefore clients do not know how to change it *directly*



## ADT: *Some* advantages

2. Even if implementation changes, as long as the interface remains stable, no changes are required in the client.

## ADT: *Some* advantages

3. With this approach it is easy to ensure *class invariants*, *e.g.*, that circunference radius will never be negative.



## ADT: *Some* advantages

4. The user of an ADT can focus on the problem domain and forget about representation and implementation details.

## ADT: *Some* advantages

5. We have now ADT that can be easily reused in other problems, and/or easily modified for some other needs.



17

## ADT: *Some* advantages

6. A program structure based on ADT is (much) more stable than another based solely on procedures/functions.



18

## Summary

- A first notion of polymorphism
- Defining characteristics of object-oriented programming

19